

Sumário

1. INTRODUÇÃO
2. DESCRIÇÃO DO PROBLEMA
3. ESTRUTURA E ESCOPO DO PROJETO
4. EXTRAÇÃO E TRATAMENTO DE DADOS
 - 4.1 Fluxo de execução
 - 4.1.1 Etapas de 1 a 3 – Abertura e extração de dados
 - 4.1.2 Etapas de 4 a 5- Separando Códigos, descrição e valores
 - 4.1.3 Etapa 6 – Limpeza e Conversão de valores numéricos
 - 4.1.4 Etapa 7 – Formato Analítico
 - 4.1.5 Etapa 8 – Exportação
5. BASES DE DADOS
 - 5.1 Tabela Dimensão Plano de Contas
 - 5.1.1 Estrutura da Tabela
 - 5.1.2 Resumo das etapas de tratamento no Power Query
 - 5.2 Tabela Fato Lançamentos DRE
 - 5.2.1 Estrutura da Tabela
 - 5.2.2 Resumo das etapas de tratamento no Power Query
 - 5.2.3 Remoção de Contas Sintéticas
 - 5.3 Tabela Calendário
6. VISUALIZAÇÃO E ANÁLISE DE DADOS COM POWER BI
 - 6.1 Tela 1 – Visão Executiva
 - 6.2 Tela 2 – Diagnóstico
 - 6.3 Principais Insights
 - 6.4 Métricas e Indicadores utilizados no Power BI
 - 6.5 Modelo e Relacionamento de dados

1. INTRODUÇÃO

A Demonstração do Resultado do Exercício (DRE) é um dos principais relatórios financeiros utilizados para avaliar a performance econômica de uma empresa em determinado período. Ela evidencia a formação do resultado, detalhando receitas, custos, despesas e lucro, permitindo analisar margens, rentabilidade e eficiência operacional.

A estrutura da DRE decompõe o caminho do topo ao fundo: partindo da Receita de Vendas o ponto de partida de qualquer análise até o Lucro Líquido do Exercício, passando por custos, despesas operacionais, resultado financeiro e tributação. Cada linha dessa jornada carrega um significado estratégico e permite identificar onde valor é criado ou destruído.

Para analistas, gestores e investidores, a DRE responde perguntas críticas: a empresa cresce com qualidade? As margens estão se expandindo ou comprimindo? Os custos estão sob controle? O resultado operacional sustenta o crescimento? Essas perguntas não podem ser respondidas com uma foto estática exigem análise temporal, comparativa e estruturada.

2. DESCRIÇÃO DO PROBLEMA

Problema central: Como transformar dados financeiros semiestruturados de um relatório público em formato PDF em um modelo analítico que permita análise de margens, evolução temporal e performance operacional com precisão, rastreabilidade e profundidade?

3. ESTRUTURA E ESCOPO DO PROJETO

Este projeto foi concebido como uma demonstração aplicada de competências em análise financeira, engenharia de dados e inteligência analítica integrando técnica e visão de negócio em todas as suas etapas.

O projeto segue cinco módulos sequenciais:

Módulo	Entrega	Demonstração
1. Extração de dados	Pipeline Python: PDF → CSV estruturado	Engenharia de dados, automação, tratamento de texto
2. Modelagem Analítica	Estrutura de dados com hierarquia contábil	Modelagem dimensional, pandas, lógica contábil
3. Definição de escopo para construção da análise e indicadores	Cálculo de margens e variações por período	Análise financeira, raciocínio sobre resultado
4. Visualização	Dashboard Power BI	Storytelling com dados, comunicação executiva

4. EXTRAÇÃO E TRATAMENTO DE DADOS

Foi desenvolvido um script Python com a finalidade de automatizar a extração de dados financeiros de PDFs de relatórios AMBEV SA, convertendo-os em um arquivo CSV estruturado e analítico.

4.1 Fluxo de execução

Etapa	O que acontece	Função
1. Abertura do PDF	O arquivo PDF é carregado na memória	pdfplumber
2. Seleção de páginas	Apenas as páginas 22–24 são processadas	pdfplumber
3. Extração de texto	O texto bruto de cada página é extraído linha a linha	pdfplumber
4. Filtragem de linhas	Apenas linhas com código contábil (ex: 3.01) são mantidas	re (regex)
5. Parse das linhas	Código, descrição e valores são separados com precisão	re (regex)
6. Limpeza de valores	Formatos numéricos brasileiros convertidos para float	pandas
7. Formato analítico	Tabela transposta para uma linha por conta/ano	pandas (melt)
8. Exportação	CSV gerado com encoding correto para Excel	pandas (to_csv)

4.1.1 Etapas de 1 a 3 – Abertura e extração de dados

O primeiro bloco do script é responsável por abrir o PDF, navegar até as páginas corretas e extrair o texto linha a linha. A biblioteca pdfplumber foi escolhida pela possibilidade de preservar o layout textual de PDFs com tabelas diferente de bibliotecas genéricas que retornam o texto em ordem de fluxo, ignorando colunas.

```
8 linhas_validas = []
9
10 with pdfplumber.open(r"C:\Projetos - Python\Projeto Portfólio II\DREAMBEV.pdf") as pdf:
11
12     paginas_alvo = [21, 22, 23] # páginas 22, 23 e 24
13
14     for i in paginas_alvo:
15         pagina = pdf.pages[i]
16         texto = pagina.extract_text()
17
18         for linha in texto.split("\n"):
19             linha = linha.strip()
20             if re.match(r"^\d+\.\d+", linha):
21                 linhas_validas.append(linha)
```

- 8. Inicializa lista para armazenar linhas válidas
- 10. Garante fechamento do arquivo mesmo em caso de erro
- 12. Define páginas a serem lidas (índices 21–23 → páginas 22–24)
- 15. Acessa página via pdfplumber (objeto Page)
- 16. Extrai texto completo da página (com quebras de linha)
- 18. Divide texto em linhas
- 19. Remove espaços em branco das linhas
- Aplica regex para manter apenas linhas com padrão numérico (ex: 3.01, 3.04.01)

4.1.2 Etapas de 4 a 5- Separando Códigos, descrição e valores

Nesta etapa foram aplicadas melhorias no processo de tratamento de dados contábeis extraídos de PDF, garantindo maior precisão e robustez. São tratadas validações de linhas vazias, captura correta do código contábil antes de transformações e remoção de percentuais que interferiam na leitura dos valores. Também aborda a extração segura dos valores monetários, a limpeza da descrição para manter apenas texto e a correção na atribuição dos valores por período, utilizando uma lógica mais confiável que evita erros com dados incompletos.

```

28 def parse_linha(linha):
29     if not linha or len(linha.strip()) == 0:
30         return pd.Series([None, None, None, None, None])
31
32     codigo_match = re.match(r"^(?!\d+)(?!\d+)", linha)
33     if not codigo_match:
34         return pd.Series([None, None, None, None, None])
35
36     codigo = codigo_match.group(1)
37     resto = linha[len(codigo):].strip()
38
39     resto_sem_pct = re.sub(r"-%", "", resto)
40
41     valores = re.findall(r"^-?\d{1,3}(?:[.]\d{3})?(?:[.]\d+)?\d+", resto_sem_pct)
42
43     valores = [v for v in valores if not re.match(r"^\d+\.\d+", v)]
44
45     descricao_temp = re.sub(r"^\d+[\d.]*%", "", resto) # tira percentuais
46     descricao_temp = re.sub(r"^\d+[\d.]*", "", descricao_temp) # tira números
47     descricao = " ".join(descricao_temp.split()).strip()
48
49
50     valor_2020 = valores[0] if len(valores) >= 1 else None
51     valor_2019 = valores[1] if len(valores) >= 2 else None
52     valor_2018 = valores[2] if len(valores) >= 3 else None
53
54     return pd.Series([codigo, descricao, valor_2020, valor_2019, valor_2018])
55
56 df_parsed = df["linha"].apply(parse_linha)
57 df_parsed.columns = ["codigo", "descricao", "valor_2020", "valor_2019", "valor_2018"]

```

- 28. Verifica se a linha está vazia, retorna Series com valores nulos
- 32. Captura o código antes de qualquer tratamento usando re.match()
- 33. Valida o código, se não encontrado, descarta a linha
- 36. Extrai o código com group(1)
- Separa o restante da linha removendo o código
- Remove percentuais com re.sub() antes de extrair valores
- Extrai valores monetários com re.findall()
- Filtra tokens que ainda parecem códigos contábeis
- Remove percentuais da descrição
- Remove números restantes da descrição
- Normaliza espaços para gerar texto limpo
- Define valores por posição com verificação de tamanho da lista
- 50. Atribui 2020, 2019 e 2018 com segurança (evita IndexError)
- 54. Retorna Series com código, descrição e valores mantendo estrutura do DataFrame

4.1.3 Etapa 6 – Limpeza e Conversão de valores numéricos

Nesta etapa foi feita a limpeza e conversão dos valores, converte cada string para **float**, tratando separadores brasileiros e valores nulos

```

77 def tratar_valor(valor):
78     if pd.isna(valor):
79         return None
80     valor = str(valor).strip()
81     if valor in ["", "-", "None"]:
82         return None
83     valor = valor.replace(" ", "")
84
85     if re.search(r"\d,\d{2}$", valor):
86         valor = valor.replace(".", "").replace(",", ".")
87     else:
88         valor = valor.replace(",", ".")
89
90     try:
91         return float(valor)
92     except:
93         return None

```

- 78. Verifica se o valor é NaN com pd.isna() → retorna None
- 80. Converte o valor para string e remove espaços nas extremidades
- 81. Trata valores vazios ("", "-", "None") → retorna None
- 83. Remove espaços internos do valor
- 85. Detecta padrão brasileiro (vírgula como decimal) via regex
- 87. Se BR: remove pontos e troca vírgula por ponto
- 89. Caso contrário (padrão americano): remove vírgulas de milhar
- 90. Tenta converter para float
- 92. Em caso de erro na conversão → captura exceção (try/except)

4.1.4 Etapa 7 – Formato Analítico

O CSV gerado tem uma linha por conta e três colunas de valor (valor_2020, valor_2019, valor_2018). Esse formato horizontal é difícil de filtrar e visualizar. A função melt() transforma para o formato vertical, com uma linha por conta por ano que é o padrão ideal para Power BI e qualquer ferramenta de BI.

```

112 df_melt = df_parsed.melt(
113     id_vars=["codigo", "descricao", "nivel_1", "nivel_2", "nivel_3"],
114     value_vars=["valor_2020", "valor_2019", "valor_2018"],
115     var_name="ano",
116     value_name="valor"
117 )
118
119 df_melt["ano"] = df_melt["ano"].str.extract(r"(\d+)")

```

- 112. Aplica melt() para transformar colunas de valores em linhas (pivot inverso)
- 113. Mantém colunas fixas com id_vars (código, descrição e níveis)
- 114. Converte colunas de anos em registros com value_vars
- 115. Cria coluna ano para identificar a origem do valor
- 116. Cria coluna valor com os dados numéricos
- 119. Extrai apenas o número do ano usando regex (remove prefixo valor)

4.1.5 Etapa 8 – Exportação

A exportação usa o parâmetro 'utf-8-sig' a diferença em relação ao UTF-8, um marcador invisível no início do arquivo que instrui o Excel a interpretar o arquivo como UTF-8. Sem ele, caracteres especiais do português (ã, ç, é) aparecem corrompidos ao abrir no Excel:

```
df_melt.to_csv("dre_tratada.csv", index=False, encoding="utf-8-sig")
print("\n CSV gerado com sucesso!")
```

Executa DataFrame para formato CSV.

5. BASES DE DADOS

5.1 Tabela Dimensão Plano de Contas

Nesta etapa foi construída a tabela dimensão d_PlanoContas com código, descrição, classificação (sintética/analítica) e hierarquia de 3 níveis (N1, N2, N3) para servir de base para relacionamentos, filtros e navegação hierárquica no dashboard de análise da DRE.

A construção da d_PlanoContas é o que transforma uma simples tabela de números em um modelo analítico navegável. Sem ela, o dashboard seria uma tabela sem contexto. Com ela, o analista consegue responder perguntas como:

Qual é o peso das Despesas Logísticas sobre a Receita Líquida em cada ano?
Como evoluíram as Despesas Gerais e Administrativas de 2018 para 2020?
Qual grupo operacional mais pressionou as margens durante a pandemia?

Essas perguntas só têm resposta precisa quando os dados de valor (fato) estão conectados a uma dimensão rica e bem estruturada. E é exatamente isso que a d_PlanoContas entrega.

5.1.1 Estrutura da Tabela

A tabela foi construída com 4 colunas e 25 linhas, cobrindo todas as contas da DRE consolidada

Coluna	Tipo	Descrição	Exemplo
Código	Texto	Código contábil hierárquico	3.04.01
Descrição	Texto	Nome da conta	Despesas com Vendas
Sintética	Inteiro	1 = conta totalizadora (não tem valor direto)	1
Analítica	Inteiro	1 = conta com valor direto (folha do nível)	0

5.1.2 Resumo das etapas de tratamento no Power Query

As etapas aplicadas ficaram registradas automaticamente no painel de Etapas Aplicadas do Power Query, garantindo rastreabilidade e reprodutibilidade do processo:

Ordem	Etapa no Power Query	O que foi feito	Por que foi necessário
1	Fonte	Conexão com o arquivo Excel	Carregar os dados brutos para o Power Query.
2	Navegação	Seleção da aba com o plano de contas	Acessar a aba correta onde os dados estão armazenados.
3	Cabeçalhos Promovidos	Primeira linha promovida como nome de coluna	Transformar a primeira linha em cabeçalhos para identificar corretamente as colunas.
4	Tipo Alterado	Tipos de dados definidos (texto, inteiro)	Garantir que cada coluna tenha o tipo correto para evitar erros e permitir operações.
5	Comprimento do Texto Inserido	Coluna com contagem de caracteres do código	Criar uma coluna auxiliar para medir o tamanho do código e apoiar a lógica condicional.
6	Coluna Condicional Adicionada	Criação da coluna N1 com lógica por comprimento	Separar a primeira parte do código (N1) com base na quantidade de caracteres.
7	Preenchido Abaixo (N1)	Fill Down aplicado na coluna N1	Propagar o valor de N1 para linhas subsequentes onde o valor estava vazio
8	Coluna Condicional Adicionada	Criação da coluna N2 com lógica por comprimento	Extrair a parte intermediária do código (N2) conforme o tamanho total do código.
9	Preenchido Abaixo (N2)	Fill Down aplicado na coluna N2	Garantir consistência dos dados preenchendo células vazias com o valor anterior.
10	Coluna Condicional Adicionada	Criação da coluna N3 com lógica por comprimento	Obter a parte final do código (N3) conforme a estrutura definida do dado.
11	Tipo Alterado (final)	Ajuste de tipos nas colunas N2 e N3 para texto	Garantir a preservação do formato e dos zeros à esquerda nas colunas tratadas.

5.2 Tabela Fato Lançamentos DRE

No modelo dimensional construído para este projeto, a *f_LancamentoDRE* é o base do modelo analítico, a tabela fato. Se a *d_PlanoContas* é o catálogo que descreve *o que são* as contas, a *f_LancamentoDRE* registra *o que aconteceu*: os valores financeiros realizados em cada conta contábil, para cada ano do período analisado.

Os dados desta tabela têm origem direta no trabalho realizado no Módulo I do projeto: o script Python que extrai, trata e estrutura os dados da DRE a partir do PDF do Relatório da AMBEV. O CSV gerado naquela etapa com código da conta, descrição, ano e valor é o que alimenta esta tabela fato.

5.2.1 Estrutura da Tabela

A tabela fato foi projetada com o mínimo de colunas necessárias para garantir desempenho, clareza e relacionamento correto com a dimensão:

Coluna	Tipo	Descrição	Exemplo
código	Texto	Chave de relacionamento com d_PlanoContas	3.04.01
descrição	Texto	Rótulo descritivo da conta (redundante, auxiliar)	Despesas com Vendas
ano	Texto	Período do lançamento — chave de filtro temporal	2018
Data	Data	Data no formato 01/01/AAAA — habilita hierarquia de datas no PBI	01/01/2018
valor	Decimal	Valor financeiro em R\$ mil — a métrica principal da análise	-6.951.417

5.2.2 Resumo das etapas de tratamento no Power Query

O CSV gerado pelo script Python foi importado diretamente no Power BI e aberto no Power Query para as etapas de limpeza e filtragem. O painel de Etapas Aplicadas registrou todas as transformações, garantindo rastreabilidade completa do processo:

Ordem	Etapa no Power Query	O que foi feito	Por que foi necessário
1	Fonte	Conexão com o arquivo CSV	Ponto de entrada dos dados extraídos pelo Python
2	Cabeçalhos Promovidos	Primeira linha promovida como nome de coluna	O CSV não tinha cabeçalho separado do conteúdo
3	Tipo Alterado com Localidade	Tipos definidos com localidade pt-BR	Garante leitura correta de decimais com vírgula (padrão brasileiro)
4	Erros Substituídos	Erros de parsing substituídos por nulo	Linhas com valor '-' no PDF geravam erros de conversão numérica
5	Tipo Alterado	Reconfirmação dos tipos de dados	Ajuste final após substituição de erros
6	Consultas Mescladas (d_PlanoContas Expandido)	JOIN com a d_PlanoContas pela coluna código	Traz a coluna 'Analítica' para uso no filtro da próxima etapa
7	Linhas Filtradas	Filtro: Analítica = 1	Remove as contas sintéticas — decisão central documentada na Seção 2
8	Coluna Duplicada	Duplicação da coluna 'ano'	Gera a base para criar a coluna Data a partir do ano
9	Colunas Renomeadas	Renomeação da duplicata para 'Data'	Clareza semântica no modelo
10	Tipo Alterado1	Coluna 'Data' convertida para tipo Date	Habilita inteligência de tempo nativa do Power BI
11	Colunas Removidas	Remoção de colunas auxiliares do JOIN	Mantém apenas as 5 colunas necessárias na tabela fato
12	Colunas Reordenadas	Ordem final: código, descrição, ano, Data, valor	Organização visual e consistência com o modelo

5.2.3 Remoção de Contas Sintéticas

Contas sintéticas, são totalizadores: seu valor é, por definição, a soma das contas analíticas abaixo delas na hierarquia. Incluí-las na tabela fato ao lado de suas contas filhas criaria um problema grave de dupla contagem.

Conta	Descrição	Tipo	Valor 2018 (R\$mil)
3.04.01	Despesas com Vendas	Sintética	-12.647.536
3.04.01.01	Despesas Logísticas	Analítica	-6.951.417
3.04.01.02	Despesas Comerciais	Analítica	-5.696.119
Soma ingênua	(3.04.01 + filhas) — resultado errado	Erro	-25.295.072
Soma correta	(somente analíticas) — resultado real	Correto	-12.647.536

Somar o valor da conta sintética 3.04.01 com o de suas filhas analíticas resultaria em quase o dobro do valor real. Por isso, a tabela fato é restrita às contas analíticas e as sintéticas são utilizadas apenas para navegação hierárquica e estrutura, via a dimensão d_PlanoContas. Esse processo foi tratado nas etapas 6 e 7 do Power Query.

5.3 Tabela Calendário

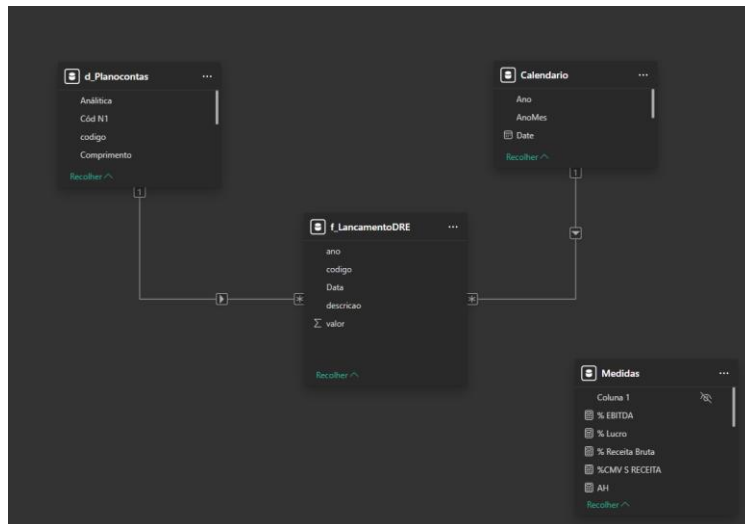
A tabela foi criada diretamente no Power BI como uma Tabela Calculada em DAX uma tabela que não vem de uma fonte externa, mas é gerada dinamicamente a partir de uma fórmula. A função CALENDAR gera uma tabela com uma única coluna (Date) contendo uma linha para cada dia entre duas datas, inclusive:

A tabela resultante possui 6 colunas, cada uma com um papel específico na análise e no comportamento do dashboard:

Coluna	Fórmula DAX	Tipo	Exemplo	Uso no dashboard
Date	CALENDAR (gerada automaticamente)	Data	2020-03-15	Chave de relacionamento com f_LancamentoDRE
Ano	YEAR([Date])	Inteiro	2020	Slicer de ano, eixo de gráfico, filtro de período
Mes	FORMAT([Date], "MMM")	Texto	Mar	Rótulo de mês abreviado em eixos e tabelas
Mes Numero	MONTH([Date])	Inteiro	3	Ordenação correta dos meses (Jan=1... Dez=12)
AnoMes	FORMAT([Date], "YYYY-MM")	Texto	2020-03	Eixo cronológico com ordenação automática correta
Trimestre	"T" & FORMAT([Date], "Q")	Texto	T1	Agrupamento e slicer trimestral

5.4 Modelo e Relacionamento de dados

Além da estrutura das tabelas, foi definido o relacionamento entre os dados com foco em performance e escalabilidade do modelo. A tabela fato de lançamentos (DRE) foi posicionada como elemento central, conectando-se às dimensões de tempo (Calendário) e plano de contas, permitindo diferentes cortes analíticos.



Essa modelagem garante:

- **Consistência nos cálculos**, evitando duplicidade ou distorções nos indicadores
- **Flexibilidade analítica**, possibilitando análises por período, conta e nível hierárquico
- **Melhor performance**, devido à separação entre fatos e dimensões

A utilização do modelo dimensional também facilita a criação de medidas em DAX, tornando os cálculos mais simples, reutilizáveis e alinhados à lógica de negócio.

6. VISUALIZAÇÃO E ANÁLISE DE DADOS COM POWER BI

Nesta etapa, realizo a definição estratégica do escopo analítico, conduzindo uma avaliação prévia dos indicadores mais relevantes para o negócio. O objetivo é estruturar uma visão clara, orientada a insights, garantindo que os dashboards desenvolvidos no Power BI atendam às necessidades reais das áreas envolvidas e gerem valor para o negócio.

6.1 Estruturação do escopo analítico e definição de indicadores

A definição do escopo analítico, que serve como base para a construção das visões em BI, segue uma lógica estruturada de geração de valor para o negócio. Esse modelo permite uma leitura completa, integrada e estratégica do desempenho, conectando os indicadores financeiros à tomada de decisão.

O processo é orientado por perguntas-chave como: “onde estamos ganhando ou perdendo eficiência?” e “o crescimento está se convertendo em resultado?” além da formulação de hipóteses de negócio que direcionam a análise. Dessa forma, a construção

dos indicadores deixa de ser apenas técnica e passa a ter um papel investigativo e estratégico.

A DRE é utilizada como principal framework analítico, pois já possui uma estrutura lógica consolidada que reflete a cadeia de geração de valor: da receita até o lucro líquido. Isso permite que a análise seja conduzida de forma padronizada, facilitando comparações e interpretações.

A partir dessa base, o primeiro passo é aplicar duas abordagens fundamentais:

Análise Vertical: avalia o peso de cada linha da DRE em relação à receita, permitindo identificar a representatividade de custos, despesas e margens na estrutura do resultado.

Análise Horizontal: avalia a evolução dos indicadores ao longo do tempo, evidenciando tendências de crescimento, redução ou estabilidade.

A combinação dessas duas análises possibilita identificar mudanças estruturais no negócio, como aumento desproporcional de custos, compressão de margens ou crescimento não sustentável.

Para responder às perguntas-chave de forma mais objetiva, é necessário isolar algumas medidas de análise vertical e horizontal e utilizá-las de forma independente no dashboard. Embora essas análises façam parte da estrutura da DRE, sua separação em indicadores específicos permite uma leitura mais clara, comparável e acionável. Ao destacar essas métricas, o usuário consegue identificar rapidamente variações relevantes sem depender exclusivamente da interpretação da tabela completa.

Com isso, o dashboard deixa de ser apenas uma representação estática da DRE e passa a atuar como uma ferramenta dinâmica de análise, onde os principais indicadores são organizados de acordo com sua relevância para a tomada de decisão.

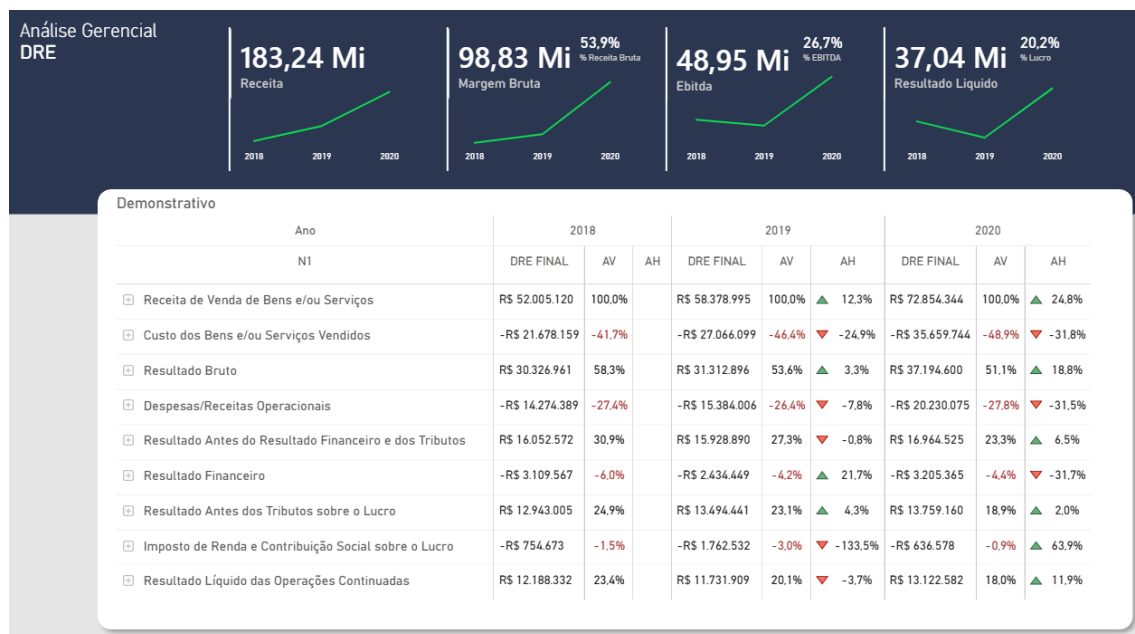
6.2 Métricas e Indicadores utilizados no Power BI

Na tabela abaixo são listados todos os indicadores utilizados para construção da visão em BI e análise dos dados.

Nome da Medida	Fórmula Completa (DAX)	O que é, e por que foi utilizado
%EBITDA	% EBITDA = DIVIDE([Ebitda], [Receita Bruta])	Mede a eficiência operacional e permite acompanhar a rentabilidade da operação em relação à receita gerada.
% Lucro	% Lucro = DIVIDE([Resultado Líquido], [Receita Bruta])	Avalia a margem líquida da operação, permitindo visualizar o resultado final em relação ao faturamento.
%Margem Bruta	% Margem Bruta = DIVIDE([Resultado Bruto], [Receita Bruta])	Mede a eficiência da operação considerando apenas o lucro bruto sobre a receita.
AH	AH = DIVIDE([DRE FINAL] - [DRE FINAL AA], ABS([DRE FINAL AA]))	Permite analisar a variação percentual em relação ao ano anterior e identificar tendências de crescimento ou queda.
AV	AV = DIVIDE([DRE FINAL], CALCULATE([DRE FINAL], ALL(d_Planocontas), d_Planocontas[Cód N1] = "3.01"))	Mede a participação de cada linha em relação à receita total, apoiando análises estruturais do DRE.
DRE	DRE = CALCULATE([Resultado], FILTER(ALL(d_Planocontas), d_Planocontas[Cód N1] < SELECTEDVALUE(d_Planocontas[Cód N1])))	Constrói a estrutura contábil do DRE de forma progressiva para visualização consolidada dos resultados.
DRE FINAL	DRE FINAL = IF(SELECTEDVALUE(d_Planocontas[Sintética]) = "0", [DRE], [Resultado])	Controla a exibição analítica e sintética das informações financeiras no dashboard.
DRE FINAL AA	DRE FINAL AA = CALCULATE([DRE FINAL], SAMEPERIODLASTYEAR(Calendario[Date]))	Permite comparações temporais com o mesmo período do ano anterior para análise evolutiva.
EBITDA	Ebitda = CALCULATE([DRE FINAL], d_Planocontas[codigo] = "3.05")	Mede o resultado operacional da empresa antes de juros, impostos, depreciação e amortização.
Receita Bruta	Receita Bruta = CALCULATE(SUM(f_LancamentoDRE[valor]), d_Planocontas[Cód N1] = "3.01")	Define a base principal de receita utilizada nos cálculos financeiros e indicadores do dashboard.
Resultado	Resultado = SUM(f_LancamentoDRE[valor])	Consolida o valor total das movimentações financeiras para compor os indicadores.
Resultado Bruto	Resultado Bruto = CALCULATE([DRE FINAL], d_Planocontas[N1] = "Resultado Bruto")	Permite avaliar o desempenho operacional bruto da empresa.
Resultado Líquido	Resultado Líquido = CALCULATE([DRE FINAL], d_Planocontas[codigo] = "3.09")	Representa o resultado final da operação após todas as despesas e custos.
% CMV s/ Receita	% CMV s/ Receita = DIVIDE([CMV], [Receita Bruta])	Mede o peso do custo das mercadorias sobre a receita, apoiando análises de eficiência operacional.
Despesas Operacionais	Despesas Operacionais = CALCULATE(SUM(f_LancamentoDRE[valor]), d_Planocontas[Cód N1] = "3.04") * -1	Permite acompanhar o total de despesas operacionais e analisar a estrutura de custos da empresa.

6.3 Resumo executivo

A tela foi construída em dois blocos que se complementam: o cabeçalho entrega velocidade de leitura, a tabela entrega profundidade. Um gestor que tem 30 segundos usa só o cabeçalho. Um analista que precisa de diagnóstico desce para a tabela.



a) Cabeçalho — 4 KPIs com sparklines

Os quatro cartões representam a cadeia completa de geração de valor: partindo da Receita (o que entrou), passando pela Margem Bruta (o que sobrou após o custo do produto), pelo EBITDA (o que sobrou após as despesas operacionais) e chegando ao Resultado Líquido (o que ficou para os acionistas). Cada cartão exibe o valor acumulado dos três anos, o percentual sobre a receita e a sparkline de evolução anual.

A relação entre os quatro cartões é a informação mais importante do cabeçalho: a Receita cresce consistentemente, mas os percentuais dos demais KPIs encolhem a cada ano. Margem Bruta cai de 58,3% para 53,9%. EBITDA cai de 30,9% para 26,7%. Resultado Líquido cai de 23,4% para 20,2%. A sparkline captura isso visualmente: a linha da Receita sobe com mais inclinação do que as linhas dos demais indicadores a empresa cresce, mas não na mesma proporção em que gera resultado.

Insight acionável: a divergência entre crescimento de receita e compressão de margem é o sinal de alerta central desta tela. Não basta crescer, é necessário investigar por que os custos crescem proporcionalmente mais rápido. Esse diagnóstico está na tabela abaixo.

b) Tabela Demonstrativo — DRE com AV e AH

A tabela reproduz a estrutura contábil da DRE com três camadas de informação para cada ano: o valor absoluto (DRE FINAL), o peso sobre a receita (AV) e a variação em relação ao ano anterior (AH). As setas verdes e vermelhas na coluna AH facilitam a leitura mas exigem atenção ao contexto: em linhas de custo, seta vermelha significa que o custo cresceu, o que é desfavorável para a margem.

As linhas mais relevantes para análise cruzada com o cabeçalho são:

Custo dos Bens Vendidos passou de -41,7% da receita em 2018 para -48,9% em 2020 alta de 7,2 pontos percentuais. Esse é o principal fator que explica a queda da Margem Bruta vista no cartão. A variação AH de -31,8% em 2020 confirma que o CPV cresceu bem acima da receita (+24,8%).

Despesas/Receitas Operacionais saíram de -27,4% para -27,8% da receita aparentemente estável, mas com AH de -31,5% em 2020, novamente acima do crescimento da receita. Esse grupo é o segundo fator da compressão do EBITDA.

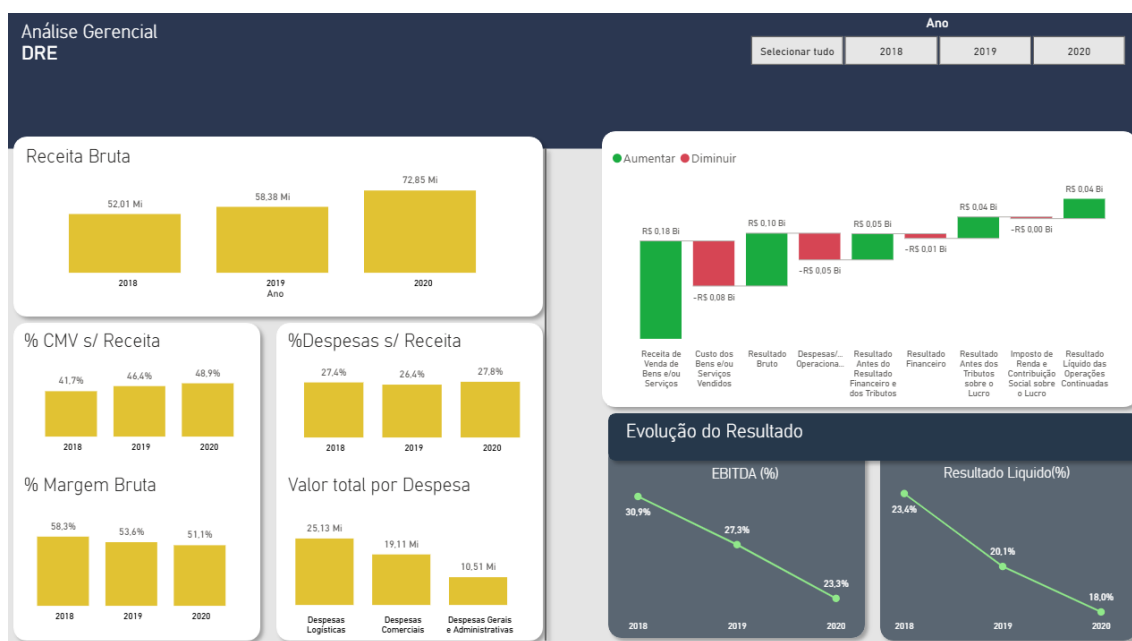
Resultado Financeiro mantém pressão negativa nos três anos, entre -4,2% e -6,0% da receita, sem melhora consistente.

Imposto de Renda apresenta uma anomalia em 2020: AH de +63,9%, indicando pagamento proporcionalmente muito menor. Esse efeito pontual é o que explica o Resultado Líquido ter crescido 11,9% em 2020 apesar de toda a compressão operacional acima. Sem esse alívio tributário, o lucro líquido teria recuado.

Insight acionável: a tabela permite identificar que a compressão de margem tem dois drivers distintos pressão estrutural no CPV (provavelmente custos de insumos e câmbio) e crescimento das despesas operacionais acima da receita. O efeito tributário de 2020 mascarou a extensão do problema no lucro líquido. A gestão precisa endereçar os dois drivers operacionais antes de depender novamente de um alívio tributário para sustentar o resultado.

6.4 Tela 2 – Diagnóstico

Enquanto a Tela 1 responde o que aconteceu, a Tela 2 responde por que aconteceu e onde está o problema. Os visuais foram organizados por dimensão analítica: volume (Receita Bruta), eficiência de custo (CMV e Margem Bruta), composição de despesas (Despesas Operacionais), formação do resultado (cascata) e tendência de rentabilidade (EBITDA% e Lucro%).



Receita Bruta: Mostra base de crescimento e referência para demais indicadores. Análise: crescimento consistente, empresa em evolução. Todos os percentuais desta tela CMV%, Margem Bruta%, EBITDA%, Lucro% são calculados sobre esses valores. Se a barra de receita cresce, mas os percentuais caem, a leitura automática é: os custos crescem ainda mais rápido que a receita.

%CMV e Margem Bruta: Os dois gráficos trabalham em par e são matematicamente complementares: a soma de CMV% e Margem Bruta% é sempre 100%. Exibi-los separados não é redundância é intenção. Ver o CMV subindo de 41,7% para 48,9% e a Margem caindo de 58,3% para 51,1% simultaneamente tem impacto visual diferente de ver apenas um deles.

A tendência é linear e consistente nos três anos, sem reversão. Isso descarta uma interpretação de evento pontual trata-se de uma deterioração estrutural progressiva.

Insight acionável: com CMV chegando a 48,9% da receita, a empresa está a menos de dois pontos de ter metade de toda sua receita consumida antes de pagar qualquer despesa operacional. O alerta é estrutural: se a trajetória se mantiver, a margem bruta de 50% piso psicológico relevante para analistas de mercado será ultrapassada no próximo ciclo. Isso exige ação nos contratos de insumos, eficiência produtiva ou revisão de mix de produtos.

Despesas Operacionais: Mostra o impacto após a Margem Bruta, evolução anual e onde está o maior gasto. Análise: Cresce proporcional a Receita e Desp. Adm com menor participação.

Insight acionável: as Despesas Logísticas crescendo 42% enquanto a receita cresce 25% é um gap de eficiência de 17 pontos percentuais. Em termos absolutos, isso representa cerca de R\$ 6 Mi de despesa adicional acima do que seria proporcional ao crescimento de receita. Esse valor, recuperado via renegociação de contratos logísticos, otimização de rotas ou mudança de modal, representaria diretamente recuperação de margem operacional.

O gráfico de cascata mostra visualmente como cada linha da DRE transforma a Receita de R\$ 183,24 Mi em Resultado Líquido de R\$ 37,04 Mi. Barras verdes representam entradas ou saldos acumulados positivos; barras vermelhas representam deduções.

A sequência visual mostra a história completa: a Receita de R\$ 183 Mi sofre a primeira e maior dedução pelo CPV (-R\$ 84 Mi, 46% da receita), resulta em Resultado Bruto de R\$ 98 Mi, sofre nova dedução pelas Despesas Operacionais (-R\$ 49 Mi, 27%), chega ao EBITDA de R\$ 48 Mi, e perde mais R\$ 8,7 Mi no Resultado Financeiro e R\$ 3,1 Mi no Imposto, chegando a R\$ 37 Mi de Resultado Líquido.

Relação com outros elementos: a cascata integra todos os visuais anteriores em uma narrativa única. O CPV que aparecia como porcentagem nos gráficos de margem aparece aqui como impacto absoluto e é de longe a maior barra vermelha. As Despesas Operacionais, segundo maior impacto, conectam-se diretamente com o gráfico de subcategorias ao lado.

Insight acionável: a leitura imediata é que CPV + Despesas Operacionais consomem 73% da receita bruta antes de qualquer efeito financeiro ou tributário. Para cada R\$ 100 de

receita, apenas R\$ 27 chegam ao resultado operacional. Qualquer iniciativa de melhoria de margem precisa endereçar primariamente esses dois blocos e dentro deles, o CPV é o maior alavancador de resultado porque representa quase o dobro do impacto das despesas operacionais.

Evolução de % EBITDA e % Lucro (linhas com fundo escuro)

Os dois painéis de linha mostram a trajetória de rentabilidade ao longo dos três anos: EBITDA caindo de 30,9% para 27,3% para 23,3%, e Lucro Líquido caindo de 23,4% para 20,1% para 18,0%. O fundo escuro não é apenas estético direciona o olhar para esses dois indicadores como os mais críticos da tela.

A inclinação da linha é a informação mais importante: em ambos os casos, a queda de 2019 para 2020 é mais acentuada do que a queda de 2018 para 2019. Isso indica que a deterioração está acelerando, não se estabilizando.

Relação com outros elementos: esses dois visuais são a consequência de tudo que os gráficos anteriores mostraram. CPV crescendo, logística crescendo, Resultado Financeiro negativo tudo se acumula aqui.

Insight acionável: a aceleração da queda de margem em 2020 especialmente no EBITDA, que perdeu 4 pontos percentuais em um único ano é o sinal de alerta mais urgente do dashboard. Se a tendência de deterioração de 3-4 pp/ano de EBITDA continuar, a empresa sairá de 30% para menos de 20% em mais dois ciclos. Esse indicador deve ser monitorado trimestralmente, com gatilho de alerta definido a partir de determinado piso sugestão de 22% como limiar de revisão estratégica.

Filtro – A visão possui filtro de Tempo Ano, que permite a análise individual por período, conforme a seleção.

6.5 Conclusão e Geração de Valor

Este projeto demonstra que é possível percorrer o caminho completo de um PDF público até uma ferramenta analítica operacional com método, rastreabilidade e foco em decisão. Mais do que um exercício técnico, o resultado entregue tem aplicação direta nos dois públicos que sustentam qualquer iniciativa de dados dentro de uma organização: o negócio e a tecnologia.

A construção do modelo permitiu transformar dados financeiros brutos em uma visão estruturada e orientada à tomada de decisão evidenciando não apenas o desempenho da AMBEV, mas principalmente os fatores que o explicam.

O principal achado analítico vai além dos números macro: a receita cresceu 25% em 2020, mas a margem bruta recuou de 58,3% para 51,1% e o EBITDA saiu de 30,9% para 23,3% no mesmo período. Crescimento isolado não garante geração de valor e o modelo torna essa distinção visível em segundos, para qualquer gestor, sem necessidade de conhecimento técnico.

Os ganhos concretos para o negócio são:

Velocidade de diagnóstico. O que exigiria horas de consolidação manual passa a estar disponível de forma permanente, atualizada e navegável. A leitura de resultado que antes dependia de alguém preparar uma planilha torna-se autossuficiente para quem toma decisões.

Clareza sobre os drivers de margem. O modelo separa o efeito do CPV, das despesas operacionais e do resultado financeiro sobre o lucro final. Gestores conseguem identificar rapidamente se uma compressão de margem vem da linha de produção, da logística ou do endividamento e direcionar ações no ponto certo.

Base para decisões estratégicas. Com AV e AH calculadas automaticamente para qualquer conta e qualquer período, o modelo apoia discussões de orçamento, revisão de metas e avaliação de desempenho sem retrabalho analítico. A pergunta *"como estamos em relação ao ano passado?"* tem resposta imediata.

Monitoramento da sustentabilidade do crescimento. O modelo evidencia se o crescimento de receita está sendo acompanhado de expansão ou compressão de margens o indicador mais crítico para avaliar a qualidade do resultado e orientar decisões de investimento e controle de custos.

Baixo custo de manutenção. Quando um novo relatório for publicado, o processo completo extração, carga e atualização do dashboard pode ser executado com ajustes mínimos. Não há lógica embutida em células de planilha, não há fórmulas frágeis dependentes de posição manual. A lógica está no código e nas medidas DAX, documentadas e visionáveis.

Replicabilidade para outros relatórios. O pipeline foi construído com parametrização em mente. Com adaptação de poucas linhas, o mesmo script pode processar relatórios de outras empresas de capital aberto disponíveis na CVM ampliando o escopo de análise sem custo proporcional de desenvolvimento.

Documentação. Cada decisão técnica do projeto está documentada com sua justificativa de negócio. Isso reduz o risco de perda de conhecimento com rotatividade de equipe e facilita a transferência do modelo para outros times ou contextos.